

Lecture 1: Introduction to Computer Organization & Assembly Language

Course

Computer Organization & Assembly Language (COAL)

Learning Objectives

By the end of this lecture, students should understand:

- What Computer Organization is
 - What Computer Architecture is
 - Difference between Architecture and Organization
 - What Assembly Language is
 - Why Assembly Language is important
 - How software communicates with hardware
 - What students will learn throughout this course
-

1. Introduction

Start your lecture with something students can relate to.



Teaching Script

Hello everyone, and welcome to the first lecture of our **Computer Organization and Assembly Language** course.

In this course, we are going to learn what actually happens inside a computer when we run a program.

We normally write programs in languages like C++, Python, Java, or JavaScript. But have you ever wondered what happens after you write your code and press the Run button?

How does the computer understand our program?

How does the CPU execute our instructions?

How are numbers, characters, and instructions stored in memory?

What exactly happens inside the processor?

And how does a programming language eventually become instructions that the CPU can execute?

These are exactly the types of questions we are going to answer throughout this course.

We will start from the fundamentals of computer organization and architecture, then move toward CPU organization, memory, instruction sets, and finally assembly language programming using MIPS.

So let's begin with the most basic question:

What is Computer Organization?

2. What is Computer Organization?

Simple Definition

Computer Organization describes **how the different hardware components of a computer are arranged and how they work together to execute instructions.**

In simple words:

Computer Organization explains **how a computer is built internally and how its components communicate with each other.**

For example, when you run a program, several components work together:

- CPU
- RAM
- Registers
- ALU
- Control Unit
- Input/Output devices
- Buses

These components don't work independently. They communicate with each other to execute your program.



Explain to Students

Let's take a simple example.

Suppose you write this C++ statement:

```
int result = 5 + 10;
```

For us, this is extremely simple. We can immediately understand that the answer is 15.

But the computer doesn't simply look at this statement and magically understand it.

The instruction has to go through several stages.

The program needs to be translated into instructions that the processor understands.

Those instructions are loaded into memory.

The CPU fetches the instructions.

The instruction is decoded.

The required operation is performed by the ALU.

And finally, the result may be stored somewhere in memory or in a register.

Understanding how all of these components work together is the purpose of Computer Organization.

3. What is Computer Architecture?

Now introduce the second important term.

Definition

Computer Architecture refers to the characteristics of a computer system that are visible to the programmer.

It includes things such as:

- Instruction Set Architecture
- Data types
- Registers
- Addressing modes
- Memory addressing

- Instruction formats

In simpler words:

Architecture describes what the computer does.

While:

Organization describes how the computer does it.

This distinction is extremely important in COAL.

4. Architecture vs Organization

This is one of the most important concepts in your first lecture.

Computer Architecture

Computer Organization

What the computer does

How the computer does it

Programmer-visible

Mostly hardware implementation

Instruction set

Hardware components

Data types

Control signals

Addressing modes

Memory technology

Registers available

CPU implementation

Instruction formats

Bus structure

Easy Example

Imagine you are buying a car.

Architecture would be things like:

- Maximum speed
- Number of seats
- Available features
- Fuel type

Organization would be:

- Engine design
- Internal components
- Wiring
- Mechanical implementation

Two cars can provide similar functionality but have completely different internal implementations.

The same concept applies to computers.

5. Why Do We Study Computer Organization?

Now explain why this subject matters.

Teaching Script

You might be wondering:

"Why do I need to study what is happening inside the computer if I can already write programs in C++, Python, or Java?"

That's a very good question.

As programmers, we don't always need to know the hardware details.

But understanding computer organization helps us write better and more efficient programs.

For example, if you understand how memory works, you can better understand why some programs use a lot of memory.

If you understand the CPU, you can understand why some operations are faster than others.

If you understand cache memory, you can understand why accessing some data is much faster than accessing other data.

And if you understand assembly language, you can see exactly how high-level programming instructions are translated into low-level CPU instructions.

So COAL helps you understand what is happening underneath your code.

6. What is Assembly Language?

Now transition into the second major topic.

Definition

Assembly language is a low-level programming language that provides a human-readable representation of machine instructions.

Computers ultimately execute **machine code**, which consists of binary instructions.

For example, a CPU works with patterns of bits such as:

010011001010...

Writing large programs directly in binary would be extremely difficult.

Assembly language provides symbolic instructions that are much easier for humans to understand.

For example:

ADD \$t0, \$t1, \$t2

This tells the processor to perform an addition using registers.

Instead of thinking in terms of raw binary, programmers can work with instructions such as:

ADD
SUB
LW
SW
BEQ
J

These are called **mnemonics**.

7. High-Level vs Low-Level Languages

Introduce the hierarchy.

High-Level Languages



C / C++



Assembly Language



Machine Code



CPU

Explain that Python, Java, C++, etc. are designed to be easier for humans.

Assembly is much closer to the hardware.

Machine code is what the processor ultimately executes.

Comparison

High-Level Language	Assembly
Easier to write	More difficult
Hardware independent	Hardware dependent
More abstraction	Less abstraction
Easier to maintain	More difficult to maintain
Usually less direct hardware control	Direct hardware-level control

8. What is Machine Language?

Before going deeper into assembly, students need to understand machine language.

Definition

Machine language is the binary instruction format directly understood by a processor.

For example:

10110000 01100001

The exact binary representation depends on the processor's instruction set.

Humans don't generally want to write programs this way.

That's where assembly language helps.

9. Assembly Language vs Machine Language

Explain the relationship:

Assembly Language



Assembler



Machine Code



CPU

Important Term: Assembler

An **assembler** is a program that translates assembly language instructions into machine code.

For example:

ADD \$t0, \$t1, \$t2

is translated by an assembler into the corresponding machine instruction encoding that the processor can execute.

10. Why Learn Assembly Language?

Give students practical reasons.

1. Understand the CPU

Assembly allows us to see how the CPU actually executes instructions.

2. Understand Memory

We can directly work with registers and memory addresses.

3. Understand Performance

We can understand how instructions affect execution time and resource usage.

4. Understand Compilers

Assembly helps us understand what happens between high-level code and machine code.

5. Embedded Systems

Assembly is still relevant in areas where low-level hardware control matters.

6. Operating Systems

Low-level concepts are important for understanding operating systems and system-level programming.

7. Cybersecurity & Reverse Engineering

Assembly knowledge is useful when analyzing compiled programs and understanding how software interacts with hardware.

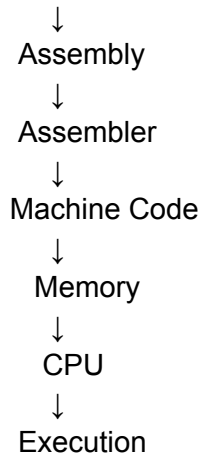
11. How Does a Program Reach the CPU?

This is a great concept to introduce in Lecture 1.

Show students this flow:

C / C++ / Other High-Level Language

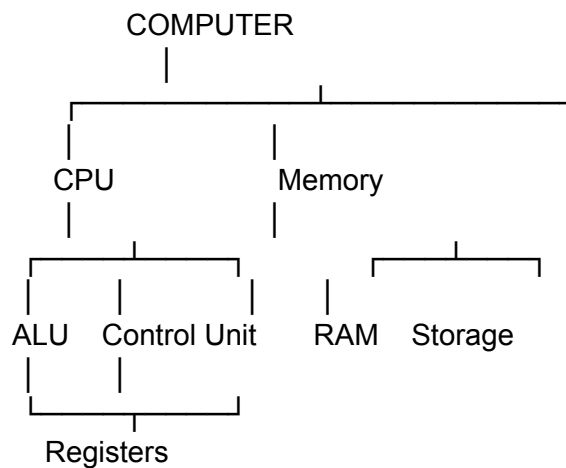
↓
Compiler



Explain that the exact compilation pipeline can vary depending on the language, compiler, platform, and optimization settings, but the key idea is that the CPU ultimately executes machine-level instructions.

12. Basic Computer Components

Now introduce the major components that students will study later.



CPU

The **Central Processing Unit** executes instructions.

ALU

The **Arithmetic Logic Unit** performs:

- Arithmetic operations
- Logical operations
- Comparisons

Control Unit

The Control Unit coordinates the activities of the CPU.

Registers

Registers are small, fast storage locations inside the processor.

Memory

Memory stores programs and data that the CPU needs.

13. What Happens When You Run a Program?

Use a very simple example.

Suppose:

```
int a = 5;  
int b = 10;  
int c = a + b;
```

Explain conceptually:

Program



Instructions



Memory



CPU fetches instruction



CPU decodes instruction

↓
ALU performs operation
↓
Result stored

This introduces the **Fetch–Decode–Execute Cycle**, which you will teach in detail later.

14. What Will We Learn in This Course?

End the lecture by showing students the roadmap.

Module 1

Computer Organization Fundamentals

Module 2

Number Systems & Data Representation

Module 3

Digital Logic

Module 4

CPU Organization

Module 5

Instruction Set Architecture

Module 6

MIPS Assembly Language

Module 7

Procedures & Stack

Module 8

Memory Organization

Module 9

Advanced CPU Concepts

Module 10

Practical Assembly Programming

Module 11

Exam Preparation
